Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.

Import Libraries

```
import pandas as pd
import numpy as np

import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Load Dataset

```
dataset = pd.read_csv('/content/diabetes_dataset.csv')
```

Explore Dataset (EDA)

```
dataset.head()
```

	year	gender	age	location	race:AfricanAmerican	race:Asian	race:Caucasian	race:Hispanic	race:Other	hypertension	heart
0	2020	Female	32.0	Alabama	0	0	0	0	1	0	
1	2015	Female	29.0	Alabama	0	1	0	0	0	0	
2	2015	Male	18.0	Alabama	0	0	0	0	1	0	
3	2015	Male	41.0	Alabama	0	0	1	0	0	0	
4	2016	Female	52.0	Alabama	1	0	0	0	0	0	

Next steps: Generate code with `dataset` New interactive sheet

```
dataset.tail()
```

	year	gender	age	location	race:AfricanAmerican	race:Asian	race:Caucasian	race:Hispanic	race:Other	hypertension	h
99995	2018	Female	33.0	Wyoming	0	0	0	0	1	0	
99996	2016	Female	80.0	Wyoming	0	1	0	0	0	0	
99997	2018	Male	46.0	Wyoming	0	1	0	0	0	0	
99998	2018	Female	51.0	Wyoming	1	0	0	0	0	0	
99999	2016	Male	13.0	Wyoming	0	0	0	1	0	0	

```
dataset.shape
```

```
(100000, 16)
```

```
print(dataset.columns)
```

```
Index(['year', 'gender', 'age', 'location', 'race:AfricanAmerican',
       'race:Asian', 'race:Caucasian', 'race:Hispanic', 'race:Other',
       'hypertension', 'heart_disease', 'smoking_history', 'bmi',
       'hbA1c_level', 'blood_glucose_level', 'diabetes'],
      dtype='object')
```

```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 16 columns):
 #   Column                Non-Null Count   Dtype  
---  ------                --------------   -----  
 0   year                  100000 non-null  int64  
 1   gender                100000 non-null  object 
 2   age                   100000 non-null  float64
 3   location              100000 non-null  object 
 4   race:AfricanAmerican  100000 non-null  int64  
 5   race:Asian            100000 non-null  int64  
 6   race:Caucasian        100000 non-null  int64  
 7   race:Hispanic         100000 non-null  int64  
 8   race:Other            100000 non-null  int64  
 9   hypertension          100000 non-null  int64  
 10  heart_disease         100000 non-null  int64  
 11  smoking_history       100000 non-null  object 
 12  bmi                   100000 non-null  float64
 13  hbA1c_level           100000 non-null  float64
 14  blood_glucose_level   100000 non-null  int64  
 15  diabetes              100000 non-null  int64  
dtypes: float64(3), int64(10), object(3)
memory usage: 12.2+ MB
```

```
dataset.describe()
```

	year	age	race:AfricanAmerican	race:Asian	race:Caucasian	race:Hispanic	race:Other	hypertensi
count	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.00000	100000.000000	100000.000
mean	2018.360820	41.885856	0.202230	0.200150	0.198760	0.19888	0.199980	0.074
std	1.345239	22.516840	0.401665	0.400114	0.399069	0.39916	0.399987	0.263
min	2015.000000	0.080000	0.000000	0.000000	0.000000	0.00000	0.000000	0.000
25%	2019.000000	24.000000	0.000000	0.000000	0.000000	0.00000	0.000000	0.000
50%	2019.000000	43.000000	0.000000	0.000000	0.000000	0.00000	0.000000	0.000
75%	2019.000000	60.000000	0.000000	0.000000	0.000000	0.00000	0.000000	0.000
max	2022.000000	80.000000	1.000000	1.000000	1.000000	1.00000	1.000000	1.000

Check Missing Values যদি থাকে

- Drop
- Mean
- Median
- Mode
- KNN Imputer

```
dataset.isnull().sum()
```

	0
year	0
gender	0
age	0
location	0
race:AfricanAmerican	0
race:Asian	0
race:Caucasian	0
race:Hispanic	0
race:Other	0
hypertension	0
heart_disease	0
smoking_history	0
bmi	0
hbA1c_level	0
blood_glucose_level	0
diabetes	0

dtype: int64

```
dataset.isna().sum()
```

	0
year	0
gender	0
age	0
location	0
race:AfricanAmerican	0
race:Asian	0
race:Caucasian	0
race:Hispanic	0
race:Other	0
hypertension	0
heart_disease	0
smoking_history	0
bmi	0
hbA1c_level	0
blood_glucose_level	0
diabetes	0

dtype: int64

Check Duplicates যদি থাকে

- df.drop_duplicates(inplace=True)

```
dataset.duplicated().sum()
```

```
np.int64(14)
```

```
# we can see there is a duplicate value, so we apply
dataset.drop_duplicates(inplace=True)
```

```
dataset.duplicated().sum()
```

```
np.int64(0)
```

Check Target Variable

```
dataset['diabetes'].value_counts()
```

diabetes	count
0	91486
1	8500

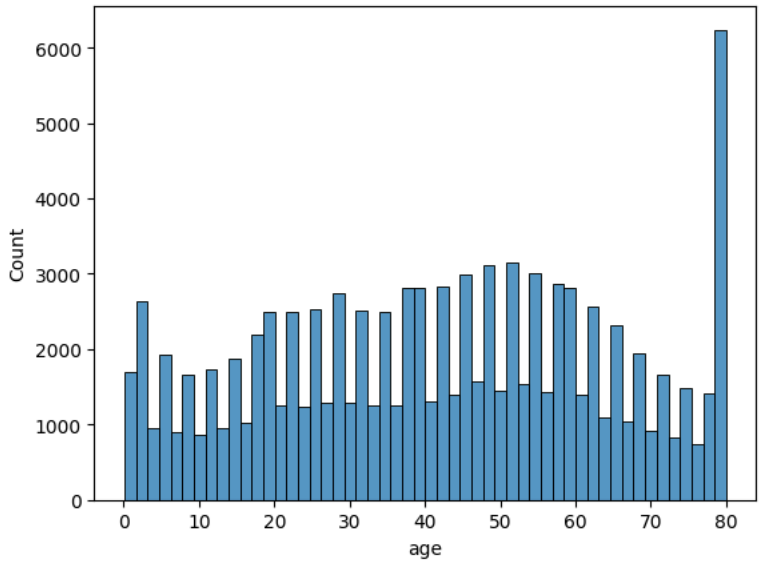
dtype: int64

```
Start coding or generate with AI.
```

Exploratory Data Analysis (EDA)

```
sns.histplot(dataset['age'])
```

```
<Axes: xlabel='age', ylabel='Count'>
```




```
# sns.heatmap(dataset.corr())
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
print(dataset["diabetes"].dtype)
print(dataset["diabetes"].unique())
print(dataset["diabetes"].value_counts())
```

```
int64
[0 1]
diabetes
0    91486
1     8500
Name: count, dtype: int64
```

Feature Selection

```
X = dataset.drop("diabetes", axis=1)
```

```
y = dataset["diabetes"]
```

```
Start coding or generate with AI.
```

Feature Engineering (Optional)

Data Preprocessing Missing Value

- SimpleImputer

Encoding যদি categorical থাকে

- OneHotEncoder
- LabelEncoder
- OrdinalEncoder

Scaling যদি প্রয়োজন হয়

- StandardScaler
- MinMaxScaler
- RobustScaler

ColumnTransformer সব preprocessing একসাথে।

- preprocessor = ColumnTransformer(...)

```
# BMI category
dataset["bmi_category"] = pd.cut(
    dataset["bmi"],
    bins=[0, 18.5, 25, 30, 100],
    labels=["Underweight", "Normal", "Overweight", "Obese"]
)
```

```
# Age Group
dataset["age_group"] = pd.cut(
    dataset["age"],
    bins=[0, 30, 45, 60, 100],
    labels=["Young", "Adult", "Middle", "Senior"]
)
```

```
# BMI × HbA1c
dataset["bmi_hba1c"] = (
    dataset["bmi"] *
    dataset["hbA1c_level"]
)
```

```
# BMI × Glucose
dataset["bmi_glucose"] = (
    dataset["bmi"] *
    dataset["blood_glucose_level"]
)
```

```
# HbA1c × Glucose

dataset["glucose_hba1c"] = (
    dataset["blood_glucose_level"] *
    dataset["hbA1c_level"]
)
```

```
# Risk Score

dataset["risk_score"] = (
      dataset["hypertension"]
    + dataset["heart_disease"]
    + (dataset["bmi"] > 30).astype(int)
)
```

```
# Age × BMI
dataset["age_bmi"] = (
    dataset["age"] *
```

```
    dataset["bmi"]
)
```

```
# Age × Glucose
dataset["age_glucose"] = (
    dataset["age"] *
    dataset["blood_glucose_level"]
)
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Start coding or generate with AI.

Identify columns types

```
num_cols = X_train.select_dtypes(include=np.number).columns
cat_cols = X_train.select_dtypes(exclude=np.number).columns
```

Preprocessing Pipeline or columns transformer

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, num_cols),
        ("cat", categorical_transformer, cat_cols)
    ]
)
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Target columns encoding

```
# Y = dataset["diabetes"]

# print("Before encoding:")
# print(Y.value_counts())

# Y = dataset["diabetes"].map({
#     "No": 0,
#     "Yes": 1
# })

# print("After encoding:")
# print(Y.value_counts())
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

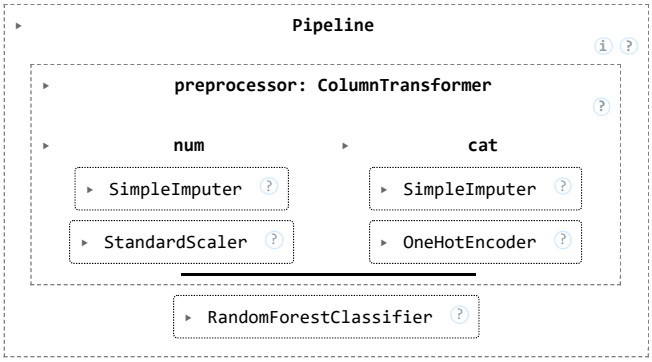
```
Start coding or generate with AI.
```

Build ML Model pipeline

```
from sklearn.pipeline import Pipeline

model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(
        random_state=42
    ))
])
```

```
model.fit(X_train, y_train)
```




```
y_pred = model.predict(X_test)
```

Random Foriest Model

```
# Random Forest Pipeline
rf_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", RandomForestClassifier(
        n_estimators=200,
        random_state=42
    ))
])

# Train the model
rf_model.fit(X_train, y_train)

# Predict on test data
y_pred_rf = rf_model.predict(X_test)

# Evaluation
print("========== RANDOM FOREST RESULTS ==========")
print("Accuracy :", accuracy_score(y_test, y_pred_rf))
```

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_rf))

print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))
```

```
========== RANDOM FOREST RESULTS ==========
Accuracy : 0.9720972097209721

Confusion Matrix:
[[18292     6]
 [  552  1148]]

Classification Report:
              precision    recall  f1-score   support

           0       0.97      1.00      0.98     18298
           1       0.99      0.68      0.80      1700

    accuracy                           0.97     19998
   macro avg       0.98      0.84      0.89     19998
weighted avg       0.97      0.97      0.97     19998
```

Linear Regression(LR)

```
log_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", LogisticRegression(max_iter=1000))
])

log_model.fit(X_train, y_train)

y_pred_log = log_model.predict(X_test)

print("LOGISTIC REGRESSION RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_log))
print(confusion_matrix(y_test, y_pred_log))
print(classification_report(y_test, y_pred_log))
```

```
LOGISTIC REGRESSION RESULTS
Accuracy: 0.9607960796079608
[[18138   160]
 [  624  1076]]
              precision    recall  f1-score   support

           0       0.97      0.99      0.98     18298
           1       0.87      0.63      0.73      1700

    accuracy                           0.96     19998
   macro avg       0.92      0.81      0.86     19998
weighted avg       0.96      0.96      0.96     19998
```

```
from xgboost import XGBClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

xgb_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", XGBClassifier(
        n_estimators=200,
        learning_rate=0.1,
        max_depth=6,
        random_state=42,
        eval_metric="logloss"
    ))
])

xgb_model.fit(X_train, y_train)

y_pred_xgb = xgb_model.predict(X_test)

print("XGBOOST RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_xgb))
print(confusion_matrix(y_test, y_pred_xgb))
print(classification_report(y_test, y_pred_xgb))
```

```
XGBOOST RESULTS
Accuracy: 0.972047204720472
[[18260    38]
 [  521  1179]]
              precision    recall  f1-score   support

           0       0.97      1.00      0.98     18298
           1       0.97      0.69      0.81      1700

    accuracy                           0.97     19998
   macro avg       0.97      0.85      0.90     19998
weighted avg       0.97      0.97      0.97     19998
```

```
from sklearn.svm import SVC

svm_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", SVC(kernel="rbf", random_state=42))
])

svm_model.fit(X_train, y_train)

y_pred_svm = svm_model.predict(X_test)

print("SVM RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print(confusion_matrix(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
```

```
SVM RESULTS
Accuracy: 0.963096309630963
[[18265    33]
 [  705   995]]
              precision    recall  f1-score   support

           0       0.96      1.00      0.98     18298
           1       0.97      0.59      0.73      1700

    accuracy                           0.96     19998
   macro avg       0.97      0.79      0.85     19998
weighted avg       0.96      0.96      0.96     19998
```

```
from sklearn.tree import DecisionTreeClassifier

dt_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", DecisionTreeClassifier(random_state=42))
])

dt_model.fit(X_train, y_train)

y_pred_dt = dt_model.predict(X_test)
```

```
print("DECISION TREE RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print(confusion_matrix(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))
```

```
DECISION TREE RESULTS
Accuracy: 0.9548954895489549
[[17857   441]
 [  461  1239]]
              precision    recall  f1-score   support

           0       0.97      0.98      0.98     18298
           1       0.74      0.73      0.73      1700

    accuracy                           0.95     19998
   macro avg       0.86      0.85      0.85     19998
weighted avg       0.95      0.95      0.95     19998
```

```
from sklearn.ensemble import ExtraTreesClassifier

et_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", ExtraTreesClassifier(
        n_estimators=200,
        random_state=42
    ))
])

et_model.fit(X_train, y_train)

y_pred_et = et_model.predict(X_test)

print("EXTRA TREES RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_et))
print(confusion_matrix(y_test, y_pred_et))
print(classification_report(y_test, y_pred_et))
```

```
EXTRA TREES RESULTS
Accuracy: 0.9676967696769677
[[18260    38]
 [  608  1092]]
              precision    recall  f1-score   support

           0       0.97      1.00      0.98     18298
           1       0.97      0.64      0.77      1700

    accuracy                           0.97     19998
   macro avg       0.97      0.82      0.88     19998
weighted avg       0.97      0.97      0.96     19998
```

```
from sklearn.ensemble import AdaBoostClassifier

ada_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", AdaBoostClassifier(
        n_estimators=200,
        learning_rate=0.1,
        random_state=42
    ))
])

ada_model.fit(X_train, y_train)

y_pred_ada = ada_model.predict(X_test)

print("ADABOOST RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_ada))
print(confusion_matrix(y_test, y_pred_ada))
print(classification_report(y_test, y_pred_ada))
```

```
ADABOOST RESULTS
Accuracy: 0.9714471447144715
[[18298     0]
 [  571  1129]]
              precision    recall  f1-score   support

           0       0.97      1.00      0.98     18298
```

```
           1       1.00      0.66      0.80      1700

    accuracy                           0.97     19998
   macro avg       0.98      0.83      0.89     19998
weighted avg       0.97      0.97      0.97     19998
```

```
from sklearn.pipeline import Pipeline
from sklearn.ensemble import VotingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Base Models
lr = LogisticRegression(max_iter=1000, random_state=42)

rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

svm = SVC(
    kernel="rbf",
    probability=True,
    random_state=42
)

# Majority Voting Ensemble
voting_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", VotingClassifier(
        estimators=[
            ("lr", lr),
            ("rf", rf),
            ("svm", svm)
        ],
        voting="hard"   # Majority Voting
    ))
])

# Train
voting_model.fit(X_train, y_train)

# Predict
y_pred_vote = voting_model.predict(X_test)

# Results
print("MAJORITY VOTING ENSEMBLE RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_vote))
print(confusion_matrix(y_test, y_pred_vote))
print(classification_report(y_test, y_pred_vote))
```

10-fold

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, cohen_kappa_score
from sklearn.model_selection import train_test_split
import numpy as np
import pandas as pd

# Assuming X, y are your features and target variable

# Initialize metrics lists to store results for each fold
accuracy_list, precision_list, recall_list, specificity_list, f1_list, kappa_list = [], [], [], [], [], []

for i in range(10):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.2,
        random_state=i,
        stratify=y
    )

    # Create Random Forest classifier
    rf_model = RandomForestClassifier(
```

```
            n_estimators=100,
            random_state=i,
            n_jobs=-1
        )

        # Train the Random Forest classifier
        rf_model.fit(X_train, y_train)

        # Make predictions on the test set
        predictions = rf_model.predict(X_test)

        # Evaluate performance metrics
        accuracy = accuracy_score(y_test, predictions) * 100
        precision = precision_score(y_test, predictions, average='macro', zero_division=0) * 100
        recall = recall_score(y_test, predictions, average='macro', zero_division=0) * 100

        cm = confusion_matrix(y_test, predictions)

        if cm.shape == (2, 2):
            tn, fp, fn, tp = cm.ravel()
            specificity = (tn / (tn + fp)) * 100 if (tn + fp) > 0 else 0
        else:
            specificity = np.nan

        f1 = f1_score(y_test, predictions, average='macro', zero_division=0) * 100
        kappa = cohen_kappa_score(y_test, predictions)

        # Append metrics to lists
        accuracy_list.append(accuracy)
        precision_list.append(precision)
        recall_list.append(recall)
        specificity_list.append(specificity)
        f1_list.append(f1)
        kappa_list.append(kappa)

        print("_____")
        print(f'Fold {i + 1}:')
        print(f'Accuracy: {accuracy:.2f}')
        print(f'Precision: {precision:.2f}')
        print(f'Recall: {recall:.2f}')
        print(f'Specificity: {specificity:.2f}' if not np.isnan(specificity) else 'Specificity: N/A')
        print(f'F1 Score: {f1:.2f}')
        print(f'Kappa: {kappa:.4f}')

    # Create DataFrame
    results_df = pd.DataFrame({
        'Fold': np.arange(1, 11),
        'Accuracy': np.round(accuracy_list, 2),
        'Precision': np.round(precision_list, 2),
        'Recall': np.round(recall_list, 2),
        'Specificity': np.round(specificity_list, 2),
        'F1 Score': np.round(f1_list, 2),
        'Kappa': np.round(kappa_list, 4)
    })

    # Add average row
    average_row = {
        'Fold': 'Average',
        'Accuracy': np.round(np.mean(accuracy_list), 2),
        'Precision': np.round(np.mean(precision_list), 2),
        'Recall': np.round(np.mean(recall_list), 2),
        'Specificity': np.round(np.nanmean(specificity_list), 2),
        'F1 Score': np.round(np.mean(f1_list), 2),
        'Kappa': np.round(np.mean(kappa_list), 4)
    }
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.